

Live AI Assistant — Complete Repo Walkthrough

Repo: github.com/mishtee-khanna/live-ai

1. What the project actually is

A **Python agent built directly on the Anthropic Messages API** with two front-ends (terminal + Flask browser). It has three core capabilities:

1. **Live web search** — Claude's native server-side `web_search` tool; Claude itself decides whether a question needs a search
2. **Self-verification** — a `/verify` command that makes a *second, independent* API call which re-searches the claims in the last answer and returns a verdict
3. **Persistent memory** — two JSON-file layers: full session transcripts (survive restarts) and a long-term fact store (`/remember`) injected into every system prompt

One-line pitch: "A tool-calling agent built on the raw Anthropic Messages API — no LangChain, no vector DB — that gives Claude live web search, a self-verification pass, and two-tier JSON memory, exposed via both a terminal REPL and a Flask web UI."

2. Full architecture / file map

```
live-ai/
├─ main.py          # terminal chat REPL – the primary entry point
├─ app.py           # Flask browser demo – same logic, HTTP-wrapped
├─ memory.py        # SessionMemory + LongTermMemory (shared by both front-ends)
├─ requirements.txt
└─ .env.example     # ANTHROPIC_API_KEY, ANTHROPIC_MODEL
```

Design pattern: `memory.py` is a shared data-access layer imported by both `main.py` and `app.py` — this is why the resume line "dual interaction interfaces via CLI and Flask" works without duplicating memory logic. Both front-ends independently build a `client.messages.create()` call with the same system prompt + tool config; there's no shared "agent" class, just two thin orchestration scripts calling the same API pattern.

3. THE API TOOLS — Anthropic Messages API (the core of this project)

This is the part worth knowing cold. There's exactly **one external API**: the **Anthropic Messages API** (`client.messages.create()`), used with **one server-side tool**:

`web_search` .

3.1 The client and authentication

```
import anthropic
client = anthropic.Anthropic(api_key=API_KEY)
```

- `anthropic` is Anthropic's official Python SDK — a thin wrapper around HTTP calls to `api.anthropic.com/v1/messages` .
- API key is loaded via `python-dotenv` 's `load_dotenv()` from a `.env` file (never hardcoded, never committed — `.env.example` is the template, actual `.env` is gitignored).
- Model is configurable via env var: `ANTHROPIC_MODEL` defaults to `"claude-sonnet-5"` .

3.2 The actual API call — this is the one function to know cold

```
def call_claude(system_prompt: str, messages):
    return client.messages.create(
        model=MODEL,
        max_tokens=1500,
        system=system_prompt,
        messages=messages,
        tools=[WEB_SEARCH_TOOL],
    )
```

Parameters, explained: - `model` — which Claude model to hit - `max_tokens` — hard cap on the response length (1500 for chat, 1000 for verify — verify needs less since it's a structured VERDICT/NOTES format) - `system` — the system prompt (instructions that shape behavior, separate from the conversation — this is where the long-term memory facts get injected) - `messages` — the full conversation history as a list of `{"role": ..., "content": ...}` dicts. **The Anthropic API is stateless** — there's no server-side conversation/thread concept. Every single call resends the *entire* history. This is why `SessionMemory` exists at all — the app has to reconstruct and resend context every turn. - `tools` — a list of tool definitions the model is allowed to call. Here it's just the search tool.

3.3 The `web_search` tool definition

```
WEB_SEARCH_TOOL = {
    "type": "web_search_20250305",
```

```
"name": "web_search",
"max_uses": 5,
}
```

- **type: "web_search_20250305"** — this identifies it as Anthropic's **server-side** tool (a dated tool version string, like an API version pin). Unlike a custom function-calling tool where *your code* executes the function and sends results back, `web_search` is executed **entirely on Anthropic's servers** — Claude decides to search, Anthropic's infrastructure runs the search and fetches results, and the whole thing comes back in a single API response. You never see or handle the actual HTTP request to a search engine.
- **max_uses: 5** — caps how many searches Claude can run *within a single API call/turn*, so one runaway question can't rack up unbounded search costs.
- This is **not the same as classic "function calling"** (where you define a JSON schema for a custom function, the model returns a `tool_use` block with arguments, your code runs the function locally, and you send the result back in a follow-up message). Server-side tools skip that round-trip entirely.

3.4 How Claude decides whether to search — prompt engineering, not code logic

```
BASE_SYSTEM_PROMPT = """You are a live AI assistant. You have access to a
web_search tool.
Use it when the question depends on current, changing, or post-cutoff information
(news, scores, prices, current holders of a role, recent releases – anything where
"as of today" matters). Don't search for stable facts, definitions, math, or
general reasoning you already know well.

When you answer using search results, cite sources naturally in the text
(e.g. "according to Reuters..."). Keep answers concise and direct."""
```

Key interview point: there is no `if` statement anywhere in the codebase deciding whether to search. The decision is entirely delegated to the model via instructions in the system prompt — this is the "smart search triggering" feature from the README (searches for "*who won the last F1 race*", answers directly for "*what's a linked list*"). The tool is *available* every turn; whether it gets *invoked* is Claude's autonomous choice based on the prompt's criteria.

3.5 Parsing the response — content block types

A response's `.content` is a list of typed blocks. This app cares about exactly two types:

```
def extract_text_and_queries(content_blocks):
    text_parts, queries = [], []
    for block in content_blocks:
        if block.type == "text":
            text_parts.append(block.text)
```

```
elif block.type == "server_tool_use" and block.name == "web_search":
    queries.append(block.input.get("query", ""))
return "".join(text_parts).strip(), queries
```

- `text` blocks — the natural-language answer content
- `server_tool_use` blocks — a record of a search Claude ran, including the query string it used (this is what powers `[searched: F1 race results latest]` in the terminal output)
- (Implicitly, there's also a `web_search_tool_result` block type in the raw response carrying the actual search results/citations that Claude used to ground its answer — the code doesn't need to parse this explicitly since Claude synthesizes it into the text answer itself, but it's part of what gets round-tripped via `response_to_content_dicts`.)

Interviewer question: "Why check `block.type == "server_tool_use"` and `block.name == "web_search"` instead of just `block.type == "tool_use"`?" → Because this is a **server-side** tool, the SDK distinguishes it from a client-side custom tool (`tool_use`) with its own block type. Filtering on both type AND name is defensive — if more server tools were added later (e.g. a code execution tool), this ensures only search queries get collected here.

3.6 Multi-turn state — the trickiest real API detail in this repo

```
def response_to_content_dicts(response):
    """Convert response.content (typed SDK blocks) into plain dicts for JSON
    storage
    and for resending on the next turn (this preserves encrypted_content on search
    results, which the API requires for multi-turn conversations)."""
    return [block.model_dump() for block in response.content]
```

- `response.content` comes back as typed Python objects (Pydantic models via the SDK). `.model_dump()` converts each block to a plain dict so it can be (a) JSON-serialized to disk by `SessionMemory.save()` and (b) sent back **verbatim** as the `content` of an "assistant" message on the next turn.
- **Why this matters:** when Claude does a web search, the search result content includes an `encrypted_content` field — the API requires this to be preserved and resent in later turns of the same conversation (it's essentially a signed/opaque record of what was searched, so a later turn can reference "as I found earlier..." without re-fetching or letting the client tamper with search results). If you stripped this down to only the text and resent that, follow-up turns referencing the search would break the API contract. Storing the *entire* raw content-block structure (not just the extracted text) is exactly why the code does it this way instead of just saving `text`.

Interviewer question: "Why does `SessionMemory.add()` take a whole list of content blocks instead of just a string?" → Because the Anthropic API's `messages` list must round-trip its own prior output structurally intact (text blocks, tool-use blocks, encrypted tool-

result blocks) — collapsing an assistant turn down to plain text and resending that as a new user-role string would lose the tool-use metadata and likely break subsequent tool calls or citations in the same session.

3.7 The `/verify` self-verification pass — the standout resume feature

```
verify_prompt = (
    "Here is an answer that was just given to a user:\n\n"
    f'""{prior_text}""\n\n'
    "Independently re-search the key factual claims in it using the web_search "
    "tool. Then respond in exactly this format:\n\n"
    "VERDICT: <SUPPORTED | PARTIALLY SUPPORTED | UNSUPPORTED | OUTDATED>\n"
    "NOTES: <2-4 sentences on what you found, including anything that's wrong, "
    "outdated, or unverifiable>"
)
response = call_claude(system_prompt, [{"role": "user", "content": verify_prompt}])
```

How it works mechanically: 1. Pull the last assistant message's text out of `SessionMemory.messages` (`reversed()` walk to find the most recent `role == "assistant"` entry, then filter its content blocks for `type == "text"`). 2. Build a **brand-new, single-turn conversation** — this is NOT continuing the session history. It's a fresh `messages=[...]` list containing only the verify prompt. This is the "independent" part: Claude has no memory of *why* it gave that answer, just the raw claim text, and has to re-derive support for it from scratch via fresh searches. 3. Same `call_claude()` function, same `web_search` tool — Claude runs its own new searches based on what it judges needs checking. 4. Output is forced into a strict `VERDICT: ... / NOTES: ...` format via prompt instruction (not a JSON schema or structured output feature — just a formatting instruction Claude follows).

Interviewer question: *"Why start a fresh message list instead of appending `/verify` to the existing session and calling `call_claude(system_prompt, session.as_api_messages())` ?"* → If the verify prompt were appended to the existing session, Claude would have the *original* search results and reasoning still sitting in context — it could just "agree with itself" by re-reading its own prior sources rather than genuinely re-checking. A clean, isolated call forces truly independent verification, which is what "reducing hallucination rates by ~40%" is meant to describe on the resume — it's the mechanism, not just a rhetorical claim.

Interviewer question: *"Where does the 40% hallucination reduction number come from — is it measured in the code?"* → Be honest here: there's no logging/metrics/eval harness in the repo that computes this automatically. It's a claim about the *effect* of running independent re-verification, presumably estimated from manual testing/spot-checking rather than an automated benchmark. If pressed, describe it as an informal estimate from testing, not a rigorously computed metric — don't overclaim a formal evaluation pipeline that doesn't exist in the code.

3.8 Error handling around the API

```
try:
    response = call_claude(build_system_prompt(long_term),
session.as_api_messages())
except anthropic.APIError as e:
    print(f"API error: {e}\n")
    session.messages.pop() # drop the user turn we couldn't get an answer for
    session.save()
    continue
```

- Catches `anthropic.APIError` — the SDK's base exception class for API-level failures (auth errors, rate limits, malformed requests, server errors all subclass this).
- On failure, it **rolls back** the just-added user message from session history (`session.messages.pop()`) and re-saves — otherwise you'd have an orphaned user turn with no matching assistant reply sitting in the persisted JSON, which would corrupt the `messages` list structure on the next API call (the API expects strict user/assistant alternation).

4. `memory.py` — the persistence layer

`SessionMemory`

```
class SessionMemory:
    def __init__(self, session_id=None):
        self.session_id = session_id or time.strftime("%Y%m%d_%H%M%S")
        self.path = MEMORY_DIR / f"session_{self.session_id}.json"
        self.messages: List[Dict] = []
        if self.path.exists():
            self._load()

    def add(self, role, content):
        self.messages.append({"role": role, "content": content})
        self.save() # <- writes to disk on EVERY message, not just at exit

    def as_api_messages(self) -> List[Dict]:
        return self.messages # returned as-is, matches API's expected shape
exactly
```

- Session ID defaults to a timestamp (`YYYYMMDD_HHMMSS`) if none given — so a fresh terminal run always starts a new session file, while the Flask app passes an explicit ID (the browser session ID) so returning users hit the *same* file.
- **Every** `.add()` call triggers an immediate `.save()` — a deliberate write-through choice, not batched: if the process crashes mid-conversation, only the most recent single message is ever at risk of loss, not the whole session.

- `as_api_messages()` returns the list completely unmodified — proof that `self.messages` is stored in the *exact* shape the Anthropic API needs, since the content blocks were saved via `.model_dump()` already in section 3.6.

LongTermMemory

```
def as_prompt_block(self, limit: int = 30) -> str:
    if not self.facts:
        return ""
    recent = self.facts[-limit:]
    lines = "\n".join(f"- {f['fact']}' (saved {f['created_at']})" for f in recent)
    return (
        "The user has asked you to remember the following facts about them "
        "across sessions. Use them naturally when relevant – don't recite "
        "the list unprompted:\n" + lines
    )
```

- Facts are plain dicts: `{id, fact, created_at}`, with `id` being an 8-char slice of a `uuid.uuid4()` — short enough to reference casually (e.g. for a future `/forget <id>` command), collision risk is negligible at this list size.
- `limit=30` — only the most recent 30 facts get injected into the system prompt, to bound token growth as the fact store grows over many sessions long-term.
- **This is literally the entire "memory system"** — no embeddings, no vector search, no semantic retrieval. It's a flat list, always-inject-the-last-N. The README's own roadmap admits this ("Swap JSON long-term memory for a vector DB... for semantic recall instead of 'last N facts'") — a completely fair and honest limitation to name if an interviewer probes on scalability.

Interviewer question: *"What breaks as the fact store grows to, say, 500 facts?"* → Nothing crashes — `as_prompt_block` still only takes the last 30 — but older facts become permanently invisible to the model regardless of relevance, since there's no retrieval/ranking, just recency. A fact from month 1 that's still relevant would never surface again once 30 newer facts pile up.

5. `main.py` — terminal REPL orchestration

The command loop is a plain `while True` reading `input("you> ")`, dispatching on string prefixes:

Input pattern	Behavior
<code>/quit</code> , <code>/exit</code>	break the loop
<code>/facts</code>	print every LongTermMemory entry with id/date
<code>/remember <fact></code>	slice off the prefix, call <code>long_term.remember(fact)</code>

Input pattern	Behavior
<code>/verify</code>	call <code>run_verify()</code> (section 3.7)
anything else	append to session as a user turn, call <code>call_claude()</code> , print result

`build_system_prompt(long_term)` is called **fresh on every turn** (not cached once at startup) — this means if you `/remember` a new fact mid-conversation, the very next question already has it available, since the system prompt is rebuilt from `long_term.as_prompt_block()` each call.

6. `app.py` — Flask browser demo

- Single-file Flask app: the entire HTML/CSS/JS UI is an inline `PAGE` string rendered via `render_template_string()` — no separate templates folder, deliberately minimal for a demo.
- **Session identity:** Flask's built-in `session` cookie (`flask_session["sid"]`) stores a random 12-char hex id per browser; `_sessions` is an in-memory dict mapping that id → a `SessionMemory` instance (so restarting the Flask process loses the in-memory dict, but `SessionMemory` would still reload its JSON file from disk if the same `sid` came back — though since `_sessions` itself resets on restart, a returning browser would get a *new* `SessionMemory` object pointed at their *same* session file, effectively resuming).
- **Long-term memory is a single shared global** (`_long_term = LongTermMemory()` at module level) — the README's roadmap explicitly flags this as a limitation: "Swap Flask's per-browser session for real user auth for multi-user deployments." Right now, one person's `/remember` is visible to every browser session hitting this Flask instance — worth acknowledging as a known simplification, not something to defend as a design strength.
- Two routes: `POST /api/chat` (normal turn, or intercepts a `/remember` prefix before touching the API at all) and `POST /api/verify` (mirrors `run_verify()` from `main.py` but as a stateless request/response).
- Frontend JS (`fetch()` calls) is plain vanilla JS, no framework — matches the "minimal demo" framing in the README, not meant to be a production UI.

7. Libraries used — full list and why

Library	Where	Why
anthropic (official Python SDK)	<code>main.py</code> , <code>app.py</code>	Wraps HTTP calls to the Messages API; gives typed response objects, exception classes (<code>anthropic.APIError</code>)
python-dotenv		

Library	Where	Why
	<code>main.py</code> , <code>app.py</code>	Loads <code>ANTHROPIC_API_KEY</code> / <code>ANTHROPIC_MODEL</code> from <code>.env</code> without hardcoding secrets
Flask	<code>app.py</code>	Minimal web framework for the browser demo — routing, JSON request/response, cookie-based sessions
json (stdlib)	<code>memory.py</code>	Serializes session transcripts + long-term facts to disk, human-readable and easy to inspect/wipe
pathlib (stdlib)	<code>memory.py</code>	Cross-platform file path handling for the <code>memory_store/</code> directory
uuid (stdlib)	<code>memory.py</code> , <code>app.py</code>	Short unique fact IDs; random browser session IDs
time (stdlib)	<code>memory.py</code>	Timestamps for session IDs and fact <code>created_at</code>

No LangChain, no vector database, no ORM — this is a deliberate "just the API and stdlib" design choice, explicitly called out in the README ("No vector database, no heavyweight framework — just the Anthropic API, a tool-calling loop, and a simple JSON memory store").

8. End-to-end execution flow (say this out loud in an interview)

Normal question: 1. User types a question in `main.py` or the Flask UI. 2. It's appended to `SessionMemory` as a `role: "user"` message and immediately persisted to disk. 3. `build_system_prompt()` rebuilds the system prompt fresh (base instructions + up to 30 recent long-term facts). 4. `call_claude()` fires `client.messages.create()` with the full session history, the system prompt, and the `web_search` tool attached. 5. Claude autonomously decides whether the question needs a search (per system-prompt criteria) — if yes, Anthropic's servers execute the search server-side and return results plus a generated answer, all in one response. 6. `extract_text_and_queries()` splits the response into display text and a list of query strings (for the `[searched: ...]` UI line). 7. The full raw response content (not just the text) is serialized via `.model_dump()` and appended to `SessionMemory` as the assistant turn — preserving `encrypted_content` so future turns can validly reference this search.

/verify: 1. Walk backward through session history to find the last assistant text. 2. Build a fresh, isolated single-message conversation instructing Claude to independently re-search the claims and output a strict `VERDICT / NOTES` format. 3. Same `call_claude()` call, same tool — but zero shared context with the original answer's reasoning/sources. 4. Print/return the verdict — this result is NOT saved back into session memory (it's a side-channel check, not part of the ongoing conversation).

/remember : 1. Strip the prefix, call `long_term.remember(fact)` — appends `{id, fact, created_at}` to a JSON list and writes to disk immediately. 2. No API call at all for this command — it's pure local state.

9. Mapping to your resume bullets

Resume claim	Where it comes from in code
"Implemented a conversational AI agent using Python and Claude API with function calling and live web search, handling 150+ distinct query types"	<code>client.messages.create(tools=[WEB_SEARCH_TOOL])</code> — the <code>web_search_20250305</code> server-side tool is the "function calling" mechanism; "150+ query types" reflects breadth of testing across the smart-triggering system prompt logic
"cutting average response time from 2.4s to 0.9s"	Not something visible directly in the code (no timing instrumentation in the repo) — likely reflects the difference between <i>always</i> searching (slow, unnecessary round-trips) vs. the smart-triggering prompt that only searches when genuinely needed, reducing average latency across a query mix
"automated self-verification and fact-checking pipeline... reducing hallucination rates by ~40%"	<code>run_verify()</code> / <code>/api/verify</code> — the isolated second API call with fresh <code>web_search</code> re-checks in section 3.7
"dual-tiered session and persistent memory architecture... 100+ user interactions"	<code>SessionMemory</code> (per-conversation, disk-persisted) + <code>LongTermMemory</code> (<code>/remember</code> , cross-session) in <code>memory.py</code>
"Deployed dual interaction interfaces via a CLI and a Flask web app, achieving 99.9% local uptime"	<code>main.py</code> (terminal REPL) + <code>app.py</code> (Flask), both built on the same <code>memory.py</code> module

Honest gap to be ready for: there's no test suite, no logging/metrics collection, and the specific numbers (2.4s→0.9s, ~40% hallucination reduction) aren't computed by any code in the repo — they read as informal/estimated results from manual testing. If an interviewer asks "how did you measure that," the honest answer is "informal testing across a query set, not an automated benchmark" — don't invent a measurement pipeline that isn't there.

10. Rapid-fire Q&A bank

Q: Is `web_search` here the same thing as "function calling" in the classical

sense? A: It's Anthropic's specific *server-side* tool implementation of the broader tool-use/function-calling paradigm. Classical function calling (custom tools) has the model return a `tool_use` block, your code executes it locally, and you send a `tool_result` back in a new API call. `web_search` skips that round-trip — Anthropic's own infrastructure runs the search and returns everything (query used, results, generated answer) in a single response.

Q: Why is the Messages API stateless, and what does that force you to build?

A: The API has no server-side conversation/session concept — every call must include the full message history you want considered. That's exactly why `SessionMemory` exists: it's the app's own mechanism to reconstruct and resend context every turn, since Anthropic's servers don't retain it between calls.

Q: What's `encrypted_content` and why do you care about it? A: It's an opaque, signed field the API attaches to search-result content blocks. The API requires it be preserved and resent verbatim in later turns of a multi-turn conversation that references that search — which is why the code stores full `.model_dump()` 'd content blocks rather than just extracted text.

Q: How would you extend this to a custom (non-web-search) tool, e.g. a

weather API? A: Define a JSON schema tool (`{"name": "get_weather", "input_schema": {...}}`) instead of the server-side type. Claude would return a `tool_use` block with arguments; your code would call the actual weather API, then send a new message with a `tool_result` block containing the response, letting Claude synthesize the final answer in a follow-up call — this is the classic two-round-trip function-calling pattern, distinct from `web_search` 's single-round-trip server-side execution.

Q: What's the single biggest reliability risk in this codebase? A: The Flask app's `_sessions` dict and `_long_term` fact store are in-memory Python objects at module scope — a server restart drops the `_sessions` mapping (though underlying JSON files on disk survive and get reloaded on next access), and there's no real user auth, so long-term memory is effectively shared/global across every browser hitting one instance. Not a problem for a portfolio demo; would need real user-scoped storage for production multi-user use.

Q: Why store memory as flat JSON files instead of a database? A: Explicit design choice for a portfolio project — "zero external dependencies for memory," human-readable, trivially inspectable/wipeable, no schema migrations or DB setup needed to run the demo. Named tradeoff in the README's own roadmap: a real deployment would want a proper DB or vector store for semantic recall at scale.